

מבני נתונים 1 - 234218

יבש 3

מגישים:

רועי קסוס - 212240956

מאור גרשמן - 322784166

איתן מיכאלוביץ' - 211878723

שאלה 1

א. עבור המחרוזת **abc**:

$S = \text{x}\text{abcy}\text{iabcz}\text{abcyr}$
123456789ABCDEF

המחרוזות שכיחה משום ש:
 $abc = S[2,4] = S[7,9] = abc$
 $xabc = S[1,4] \neq S[6,9] = iabc$
 $abcy = S[2,5] \neq S[7,A] = abcz$

עבור המחרוזת **abcy**:

$S = \text{x}\text{abcyi}\text{abc}\text{zabcyr}$
123456789ABCDEF

המחרוזות שכיחה משום ש:
 $abcy = S[2,5] = S[B,E] = abcy$
 $xabcy = S[1,5] \neq S[A,E] = zabcy$
 $abcyi = S[2,6] \neq S[B,F] = abcyr$

עבור המחרוזת **abcyi**:

$S = \text{x}\text{abcyi}\text{abczabcyr}$
123456789ABCDEF

המחרוזת איננה שכיחה משום שקיים מופע יחיד של $abcyi = S[2,6]$.

עבור המחרוזת **xabcy**:

$S = \text{x}\text{abcyi}\text{abczabcyr}$
123456789ABCDEF

המחרוזת איננה שכיחה משום שקיים מופע יחיד של $xabcy = S[1,5]$.

ב. הגדרות:

בהינתן מחרוזת S , נסמן את S^{-1} להיות המחרוזת ההפוכה של S .

בהינתן תת-מחרוזת a , נגדיר את המושגים הבאים:

1. a מקיימת את תנאי ימין, אם קיימים לפחות שני מופעים שלה ב S כך שהאינדקס של התו האחרון שלהם הוא j_1, j_2 ומתקיים $S[j_1+1] \neq S[j_2+1]$.
2. a מקיימת את תנאי שמאל, אם קיימים לפחות שני מופעים שלה ב S כך שהאינדקס של התו הראשון שלהם הוא i_1, i_2 ומתקיים $S[i_1-1] \neq S[i_2-1]$.

מבנה הנתונים:

נשמור במבנה הנתונים שלנו את S , S^{-1} ושני עצי סיפות דחוסים.

נשמור גם מספר שיהווה את אורך המחרוזת השכיחה הארוכה ביותר.

עץ 1 – יהיה של S .

עץ 2 – יהיה של S^{-1} .

כפי שראינו בהרצאה, כמות הצמתים בעץ סיפות דחוס של מחרוזת בגודל $|S|$ הוא $O(|S|)$.
ולכן בסה"כ סיבוכיות המקום שלנו תהיה $O(|S|)$.

הפעולות הנדרשות:

Init:

נבנה את עץ 1 באמצעות שימוש באלגוריתם שהוזכר בהרצאה.

כמו כן, נהפוך את S ונשמור אותה ונבנה את עץ 2 איתה באותו אופן.

נחשב ונשמור את אורך המחרוזת השכיחה ביותר כפי שיתואר בהמשך.

בסה"כ נבצע הפיכת מחרוזת ב $O(|S|)$, ועוד פעמיים בניית עץ סיפות ב $O(|S|)$.

כמו כן, חישוב המחרוזת השכיחה גם כן יקח $O(|S|)$ ולכן בסה"כ סיבוכיות זמן הריצה $O(|S|)$.

Check:

נחפש את a בעץ 1.

אם לא מצאנו אותה, אז היא לא מופיעה בעץ ולכן בהכרח איננה תת-מחרוזת קיימת (כפי שראינו בהרצאה) ולכן בפרט היא איננה תת-מחרוזת שכיחה.

אם מצאנו אותה בצומת מנוון, אז בכל מופע שלה בהכרח התו הבא היה אותו אחד (והוא התו שמופיע בקשת שיוצאת מהצומת המנוון שמצאנו), ולכן תנאי ימין בהכרח לא מתקיים ולכן בפרט היא איננה תת-מחרוזת שכיחה.

נניח שמצאנו אותה בצומת לא מנוון ולכן היא מקיימת את תנאי ימין (כי יש לפחות שני בנים לצומת – יש לפחות שני תווים שונים שמגיעים אחריה, וכמובן שיש לפחות שני עלים בתת העץ הנפרש).

כעת נחפש את a^{-1} בעץ 2.

היא בהכרח אמורה להיות בעץ מכיוון שווידאנו שזאת תת-מחרוזת קיימת.

אם מצאנו אותה בצומת מנוון, אז באופן דומה, תנאי ימין של a^{-1} בהכרח לא מתקיים ולכן תנאי שמאל של a לא מתקיים ולכן בפרט היא איננה תת-מחרוזת שכיחה.

נניח שמצאנו אותה בצומת לא מנוון ולכן היא מקיימת את תנאי שמאל (a^{-1} מקיימת את תנאי ימין כי יש לפחות שני בנים לצומת – יש לפחות שני תווים שונים שמגיעים אחריה, וכמובן שיש לפחות שני עלים בתת העץ הנפרש).

אם מצאנו את שתיהן, גם תנאי ימין וגם תנאי שמאל מתקיימים ולכן המחרוזת בהכרח שכיחה.

בסה"כ חיפוש תת-המחרוזת בכל עץ יקח $O(|a|)$, ונעשה זאת לכל היותר פעמיים ולכן סיבוכיות זמן הריצה היא $O(|a|)$.

GetLongest: נחזיר את אורך המחרוזת הארוכה ביותר ששמרנו בזמן בניית המבנה, וזה כמובן $O(1)$.

אלגוריתם חישוב המחרוזת הארוכה ביותר:

נשתמש פה רק בעץ 1 (עץ הסיפות הדחוס של S) ולו נקרא "העץ".
כעת נחפש את הענף הארוך ביותר מהשורש שמסתיים בצומת שאיננו עלה.
נבצע סריקת postorder ונספור כמה תווים עברנו מהשורש עד הצומת הנוכחי.
בכל פעם נדאג לעדכן את המשתנה של אורך המחרוזת הארוכה ביותר בהתאם לצורך.

כעת אנחנו יודעים מה אורך תת-המחרוזת הארוכה ביותר שמקיימת את תנאי ימין.
נבחר את אחת מתתי-המחרוזות הללו ונסמנה X .

נניח בשלילה ש X איננה מקיימת את תנאי שמאל.
כלומר ניתן להוסיף ל X תו כלשהו משמאל, והיא עדיין תקינה את תנאי ימין.
זאת סתירה, משום שאילו היינו מוסיפים ל X תו כלשהו משמאל והיא עדיין הייתה מקיימת את תנאי ימין, היא הייתה מופיעה בעץ עם אורך ארוך ב 1 ממה שמצאנו, ואורך X לא היה המקסימום.

לפיכך, המחרוזת הארוכה ביותר שמקיימת את תנאי ימין בהכרח גם מקיימת את תנאי שמאל ולכן היא שכיחה.

הסיבוכיות של הפעולה הזאת היא $O(|S|)$, מכיוון שבעץ סיפות דחוס יש $O(|S|)$ צמתים (ראינו בהרצאה) וסריקת postorder היא לינארית ביחס לכמות הצמתים.

שאלה 2

א. נגדיר אלגוריתם למציאת ה-k המינימלי בעבורו המערך k מונוטוני :

1. אפס משתנה מונה לאפס
2. עבור כל i בין אפס ל-1-n (לא כולל) :

אם $a[i] > a[i+1]$:

הגדל את המשתנה המונה ב-1.

3. החזר את המשתנה המונה + 1.

נכונות האלגוריתם : האלגוריתם עובר על איברי המערך, בכל פעם שמוצא אי סדר ביניהם, תת המערך שממנו התחלנו כבר לא ממוין, לכן הסתיים ומתחיל תת מערך ממוין חדש, ולכן נוסיף אחד למונה. לבסוף נוסיף אחד כדי לספור את תת המערך הממוין האחרון שלא נספר מכיוון שלא נמצא אי סדר בו. סה"כ נקבל את המספר המינימלי של תתי מערכים מסודרים בתוך המערך המקורי.

ניתוח סיבוכיות : האלגוריתם מבצע לולאה אחת על איברי המערך, ובכל איטרציה מבצע פעולת השוואה והגדלת מונה במידת הצורך שהן פעולות $O(1)$, ועל כן סיבוכיות זמן הריצה של האלגוריתם היא $O(n)$ כאשר n הוא גודל המערך.

3. נגדיר אלגוריתם למיון המערך בתנאים הנתונים :

1. נמצא את ה-k המינימלי שעבורו המערך הוא k מונוטוני באמצעות האלגוריתם מסעיף א'.

2. נגדיר מערך עזר Indexes באורך k, נגדיר $Indexes[0] = 0$ ומשתנה מונה ל-1. נבצע סריקה לינארית של איברי המערך עם משתנה רץ i, בכל פעם שנמצא אי סדר כפי שתואר בסעיף א, נכניס את i+1 למערך בגודל k במקום המשתנה המונה ונגדיל את המשתנה המונה ב-1. בסוף הלולאה נקבל מערך בגודל k, בו האינדקסים של תחילת k תתי המערכים הממוינים.

3. נגדיר מערך חדש B בגודל n על מנת למיין לתוכו את האיברים בצורה הבאה :

בעבור כל i בין 0 ל-n (לא כולל) :

א. נמצא את האיבר המינימלי מבין האיברים המתוארים על ידי האינדקסים ב-Indexes. נשמור את האינדקס של תת מערך זה במערך Indexes. פעולה מציאת מינימום במערך ידועה ולוקחת זמן לינארי בגודל המערך, גודל מערך אינדקסים הוא k.

ב. נכניס אותו למערך B במקום i.

ג. נגדיל את האיבר במערך Indexes התואם באחד.

(נדאג לכך שאם הגענו לתחילת מערך אחר, נפסיק להתחשב במקום זה במערך Indexes, לדוגמא באמצעות מערך דגלים ושמירת העתק מקורי של Indexes כך שנדע את תחילת התתי מערכים בכל רגע נתון, אלו בדיקות פשוטות שיבוצעו ב $O(1)$ בכל איטרציה).

4. נעתיק את B אל תוך המערך המקורי וכעת הוא יהיה ממוין.

נכונות האלגוריתם : כל תת מערך הוא ממוין, ועל כן בכל תת מערך האיבר המינימלי ביותר נמצא בתחילתו, בכך שאנו משווים בין כל האיברים המינימליים ביותר בכל תת מערך ומקבלים את המינימום, נקבל את האיבר המינימלי במערך כולו, ונכניס אותו למקום הראשון במערך הממוין, ונגדיל את האינדקס ההתחלתי של תת המערך הרלוונטי ("נמחק" את האיבר שהשתמשנו בו מתת המערך). אפשר להבין כי באופן אינדוקטיבי בכל איטרציה i אנו מקבלים את האיבר ה- $i+1$ הכי קטן במערך ועל כן בסופו של דבר נקבל מערך ממוין.

ניתוח סיבוכיות : שלב א רץ בסיבוכיות זמן $O(n)$ כפי שהוכח בסעיף א', שלב ב מבצע פעולות $O(1)$ בסריקה לינארית של איברי המערך ולכן גם הוא רץ בסיבוכיות זמן $O(n)$, שלב ג' רץ בסיבוכיות זמן $O(nk)$ מכיוון שבכל איטרציה הוא מבצע חיפוש מינימום במערך שאורכו k שהיא פעולה של $O(k)$ ועוד פעולות $O(1)$, ושלב ד' היא פעולת העתקת מערך שהיא $O(n)$. סה"כ קיבלנו כי סיבוכיות זמן הריצה של האלגוריתם היא :

$$O(n) + O(n) + O(nk) + O(n) = O(nk) = O(n)$$

מכיוון שנתון כי $k = O(1)$.

4. נפריך את הטענה :

נניח בשלילה שניתן לממש את $\text{MakeMonotone}(A)$ בסיבוכיות זמן ריצה של $O(n \log \log n)$. יהי מערך A בגודל n , נבצע עליו את הפעולה MakeMonotone ונקבל מערך $\log \log n - \text{monotone}$, כעת נכניס את מערך זה לאלגוריתם המיון שלנו מסעיף ג', הוכחנו בסעיף ג' כי אלגוריתם המיון שלנו רץ בזמן $O(nk)$ וידוע לנו כי $k = \log \log n$, על כן סיבוכיות זמן הריצה של האלגוריתם מסעיף ג' תהיה $O(n \log \log n)$. התחלנו ממערך A בגודל n כלשהו, והראנו כי ניתן באמצעות הפעולה MakeMonotone ואלגוריתם המיון שהצענו בסעיף ג לבצע מיון בסיבוכיות זמן ריצה של $O(n \log \log n) + O(n \log \log n)$ $O(n \log \log n)$, וזוהי **סתירה** לחסם התחתון $\Omega(n \log n)$ לאלגוריתם מיון מבוסס השוואות על מערך A כללי שהוכחנו בהרצאה מכיוון ש $O(n \log \log n)$ הוא בפרט $o(n \log n)$.

שאלה 3

א.

CanBeRouted(1,2,3,3): false (אין אף נקודה נוספת עם איקס=1 או וואי=2 במבנה)

CanBeRouted(1,2,6,1): false (אין אף נקודה נוספת עם איקס=1 או וואי=2 במבנה)

CanBeRouted(2,5,6,4): false (הנקודה (2,5) לא נמצאת במבנה)

CanBeRouted(5,4,6,1): true (קיים מסלול בין (5,4) לבין (6,4) לבין (6,1))

ב. הפתרון שלנו מסתמך על ההנחה בסעיף ב שכל הנקודות שאנחנו מקבלים תקינות.

נחזיק שני מבני UF. אחד עבור ערכי X והשני עבור ערכי Y, של כל הנקודות שקיימות לנו במערכת.

כל מבנה UF שומר מאחורי הקלעים מערך בגודל m, ובכל כניסה במערך יש מצביע לחולייה בעץ הפוך. נניח שאנחנו מבצעים איחוד לפי גודל וכיווץ מסלולים. בסה"כ נחזיק לכל מבנה כזה, לכל היותר, מערך בגודל (m+1) ו-(m+1) חוליות. זה משום שבתחום [0,m] יש m+1 מספרים טבעיים (ערכים אפשריים לערכים שלנו). לכן בסה"כ סיבוכיות המקום של המבנה שלנו הוא $O(m)$.

באופן כללי: בכל חולייה (שמייצגת ערך x או y) נחזיק מצביע לחוליית האב שלה (אם קיים), ואם החולייה היא בעצמה חוליית אב, אז נחזיק "קישור" - מצביע לשורש של עץ הפוך אבל של ערך מקביל (y או x בהתאמה).

Init: נקצה את שני המערכים, עם ערכי null. כפי שראינו בכיתה, ניתן לבצע זאת ב $O(1)$, וזה בפרט $O(m)$.

InsertPoint: נניח שקיבלנו נקודה (x,y). נחפש את x בUF של ה-x-ים. אם ערך ה-x הזה לא קיים במערכת, ניצור חולייה חדשה nodeX ונשמור אותה במערך במקום המתאים (באינדקס x). אם הנקודה כן קיימת, החיפוש יחזיר לנו את השורש של העץ ההפוך ולו נקרא nodeX. נבצע בידיוק את אותו הדבר עבור y.

אם יצרנו כעת שתי חוליות חדשות: נקשור את nodeX, nodeY אחד לשני. נשים לב שnodeX, nodeY הם שורשים של עצים הפוכים (היות ושני העצים ההפוכים האלה בעלי איבר יחיד).

אם שתי החוליות היו כבר קיימות: עבור nodeX, נשמור את החוליה שמקושרת אליו ב y ונקרא לה linkY, וכנ"ל עבור x. נשים לב שnodeX, nodeY, linkX, linkY הם כולם שורשים של עצים הפוכים לפי בניית ושימור הסדר של מבנה הנתונים שלנו. כעת נבצע איחוד בין nodeX, linkX ונבצע איחוד בין nodeY, linkY. נקרא לכל שורש חדש כזה newNodeX, newNodeY. כעת נקשור בין newNodeX, newNodeY אחד לשני.

אם אחת החוליות כבר הייתה קיימת והשנייה לא: בה"כ $nodeX$ כבר הייתה קיימת ו $nodeY$ חדשה.

$nodeX$ הוא שורש של עץ הפוך ולכן שומר את $linkY$ (שגם הוא שורש).
כעת נגדיר את $linkY$ להיות האב של $nodeY$. (ע"י עדכון מצביע האב ב $nodeY$).

מימוש זה נכון משום שברגע שאנחנו מכניסים איזשהי נקודה X או Y שלה כבר קיים במערכת, כעת כל הנקודות בעלות אותו X או אותו Y נמצאות באותו "רכיב קשירות", וניתן להגיע מאחת לשנייה, לכל שתי נקודות. לכן אנחנו גם שומרים את ה"קישור" בכל חוליה – על מנת שנוכל להגיע גם לקבוצה של הציר המאונך, וגם שם לבצע את אותו איחוד רלוונטי.

כעת ננתח את הסיבוכיות. ביצענו שני חיפושים ב UF , ולכל היותר הקצינו שתי חוליות חדשות, ושינינו שני מצביעים. בסה"כ סיבוכיות זמן הריצה של הפעולה היא $O(\log^* m)$ בשערוך עם $CanBeRouted$ (היות ואנחנו מבצעים איחוד לפי גודל וכיווץ מסלולים).

CanBeRouted: נחפש את $x1$ במבנה ונשמור את חולייתו כ $nodeX1$. הנחנו שהנקודה נמצאת במבנה ולכן בהכרח קיבלנו שורש של עץ הפוך. נעשה אותו דבר עבור $x2$.
אם $nodeX1 = nodeX2$, קיים ביניהם מסלול, ואם לא אז אין.

זה נכון מפני ששתי נקודות שנמצאות באותה קבוצת X ים בהכרח נמצאות באותו "רכיב קשירות" בזכות האופן שבו ביצענו את $InsertPoint$. מכיוון שהן באותה קבוצת X ים, יש להן את אותו השורש, ואותו אנחנו משווים. זה בהכרח נכון גם עבור y , ולכן אין חשיבות לבדיקה כזאת בגלל האנלוגיה.

ביצענו שני חיפושים ב UF , ולכן סיבוכיות זמן הריצה של הפעולה היא $O(\log^* m)$ בשערוך עם $InsertPoint$ (היות ואנחנו מבצעים איחוד לפי גודל וכיווץ מסלולים).

